

Project Name: Amazon E-commerce
Database Manageme...



Project Name: Amazon E-commerce
Database Management System

SUBTITLE: Database Design, SQL Analysis & Business
Reporting

Developed By: Kareena Gola

Technology: PostgreSQL,
pgAdmin4, SQL

ABSTRACT

ABSTRACT

This project designs and implements a relational database for an Amazon-like e-commerce platform using PostgreSQL. The database manages customers, sellers, products, orders, payments, shipments, reviews, wishlists, inventory, shopping carts, and gifts. The project demonstrates database design, normalization, relationships, constraints, and SQL querying.

Introduction

E-commerce platforms require efficient databases to manage millions of products and transactions. This project simulates an online shopping platform similar to Amazon by creating a relational database that stores customer, seller, product, order, payment, shipment, and review information.

Overview of Amazon



Overview of Amazon

Amazon is one of the world's largest e-commerce and technology companies. It was founded by Jeff Bezos on July 5, 1994, in the United States. Initially, Amazon started as an online bookstore, but over time it expanded into a global online marketplace offering millions of products across categories such as electronics, clothing, groceries, books, home appliances, and more.

It is best known as the world's biggest online shopping [retailer](#) and [marketplace](#), offering a wide range of products and services. Amazon sells [books](#), [electronics](#), [pharmaceuticals](#), [car parts](#), [video games](#), [dining](#) products, [jewelry](#), [food](#), [music](#), and [sports equipment](#).

Amazon connects customers with sellers through a secure online platform. Customers can search for products, compare prices, add items to their shopping cart, place orders, make online payments, track deliveries, and provide ratings and reviews. Sellers can list products, manage inventory, process orders, and reach customers worldwide.

In addition to e-commerce, Amazon provides services such as Amazon Prime, Amazon Web Services (AWS), Prime Video, Kindle, and Amazon Music. These services have made Amazon one of the most valuable technology companies in the world.

The success of Amazon depends on an efficient database system that manages customer information, product catalogs, orders, payments, shipping, reviews, and inventory. A well-designed database ensures accurate data storage, quick order processing, secure transactions, and a smooth shopping experience.



Overview of Amazon

This project focuses on designing a relational database for an Amazon-like e-commerce platform using PostgreSQL. The database will include entities such as Customers, Sellers, Products, Categories, Orders, Payments, Reviews, and Shipping. The objective is to understand how real-world e-commerce platforms organize and manage data while applying database concepts such as normalization, primary keys, foreign keys, relationships, and SQL queries.

Problem Statement

PROBLEM STATEMENT

Problem Statement

In today's digital world, millions of customers shop online every day. An e-commerce platform like Amazon must manage a huge amount of data, including customer information, seller details, products, orders, payments, deliveries, reviews, and inventory. If this data is not organized properly, it can lead to slow performance, incorrect orders, duplicate records, inventory issues, and poor customer experience.

A well-designed database is essential to store and manage this information efficiently. It should maintain data accuracy, reduce redundancy, ensure secure transactions, and support fast retrieval of information. The database must also handle relationships between customers, products, orders, payments, and sellers while maintaining data integrity.

This project aims to design a relational database for an Amazon-like e-commerce platform using PostgreSQL. The database will provide a structured way to manage business operations, improve data consistency, and support analytical queries for business decision-making.

Objective

Objectives of the Project

1. Design a normalized relational database for an Amazon-like e-commerce platform.
2. tables with appropriate Primary Keys and Foreign Keys.
3. Establish relationships between entities such as Customers, Products, Orders, and Payments.
4. Store and manage data efficiently while reducing redundancy.
5. Perform SQL queries to analyze sales, customers, products, and orders.
6. Generate an ER Diagram to visualize the database structure.
7. Apply SQL concepts such as joins, aggregate functions, subqueries, views, and constraints.

What is Product Dissection

Product Dissection

What is Product Dissection?

Product dissection is the process of studying a product or platform to understand its features, how users interact with it, and how different components work together. In this project, the Amazon platform is analyzed to identify its core business functions and design a database that supports those functions.

The following are the major features of an Amazon-like e-commerce platform.

1. User Registration and Login

Customers create an account by providing personal details such as name, email, phone number, and password. After registration, they can securely log in to access their account, view previous orders, manage addresses, and continue shopping.

Database Tables:

Customers

Addresses

2. Product Catalog

Amazon offers millions of products organized into categories such as Electronics, Clothing, Books, and Home Appliances. Each product contains information like name, description, price, brand, stock availability, and seller details.

Database Tables:

Products

Categories

Sellers

Inventory

3. Product Search and Filtering

Customers can search for products using keywords and apply filters such as category, brand, price range, customer rating, and availability. This helps users quickly find the products they need.

Database Tables:

Products

Categories

4. Shopping Cart

Customers can add one or more products to their shopping cart before completing the purchase. They can also update quantities or remove products.

Database Tables:

Cart

Cart_Items

Products

5. Order Management

After checkout, an order is created containing product details, quantity, order date, and current order status (such as Pending, Shipped, or Delivered).

Product Dissection

Database Tables:

Orders

Order_Items

6. Payment System

Customers can pay using different payment methods, including credit cards, debit cards, UPI, net banking, or cash on delivery. Payment records are stored for every order.

Database Tables:

Payments

Orders

7. Shipping and Delivery

Once payment is confirmed, the order is prepared for shipment and delivered to the customer's address. The customer can track the shipment status.

Database Tables:

Shipments

Addresses

Orders

8. Ratings and Reviews

After receiving the product, customers can rate it and write reviews. These reviews help future buyers make informed purchasing decisions.

Database Tables:

Reviews

Customers

Products

Conclusion of Product Dissection

The product dissection shows that Amazon is built on several connected business processes, including customer management, product management, order processing, payment handling, shipping, and customer feedback. Designing a relational database for these processes ensures efficient data management, minimizes redundancy, and supports accurate business operations.

An e-commerce platform like Amazon
serves million...

Real-World Business Problems and Solutions

An e-commerce platform like Amazon serves millions of customers and sellers every day. Managing such a large platform involves solving many business challenges. The following are some common problems and the solutions provided by Amazon.

Problem 1: Difficulty in Finding the Right Product

Problem: Customers often struggle to find the exact product among millions of available items.

Solution: Amazon provides a powerful search engine with filters based on category, price, brand, ratings, and availability. This helps customers quickly find products that match their needs.

Database Tables Used:

Products

Categories

Problem 2: Managing Large Product Inventory

Problem: Sellers need to keep track of product stock to avoid selling unavailable items.

Solution: Amazon maintains an inventory management system that updates stock levels whenever products are added, purchased, or restocked.

Database Tables Used:

Products

Inventory

Sellers

Problem 3: Secure Order Processing

Problem: Customers expect their orders to be processed accurately without missing or duplicate items.

Solution: Amazon records each order with detailed information such as products, quantity, payment status, and shipping status.

Database Tables Used:

Orders

Order_Items

Payments

Problem 4: Safe and Reliable Payments

Problem: Online transactions must be secure and accurately recorded.

Solution: Amazon supports multiple payment methods and stores payment information securely while tracking payment status.

Database Tables Used:

Payments

Orders

Real-World Business Problems and Solutions

Problem 5: Building Customer Trust

Problem: Customers want to know whether a product is good before purchasing it.

Solution: Amazon allows verified buyers to rate products and write reviews, helping other customers make informed decisions.

Database Tables Used:

Reviews

Customers

Products

Problem 6: Fast Delivery and Order Tracking

Problem: Customers want to know where their order is and when it will arrive.

Solution: Amazon provides shipment tracking and delivery status updates from dispatch to delivery.

Database Tables Used:

Shipments

Orders

Addresses

Conclusion

By addressing these real-world challenges, Amazon delivers a reliable and user-friendly shopping experience. A well-designed relational database is the foundation that supports these business operations by maintaining accurate data, reducing redundancy, ensuring data integrity, and enabling efficient order processing.

Case Study: Purchasing a Smartphone on Amazon

CASE STUDY

Case Study: Purchasing a Smartphone on Amazon

Background

A customer named Priya Sharma wants to purchase a smartphone from Amazon. She already has an Amazon account and prefers online shopping because it is convenient, offers multiple payment options, and provides home delivery.

This case study demonstrates how different features of the Amazon platform work together and how the database stores information at each stage of the shopping process.

Step 1: Customer Login

Priya logs into her Amazon account using her registered email address and password.

Database Activity: The system verifies the customer's login credentials and retrieves her account details.

Tables Used:

Customers

Step 2: Product Search

Priya searches for "Samsung Galaxy S25" using the search bar.

The system displays products based on keywords, category, price, ratings, and availability.

Tables Used:

Products

Categories

Inventory

Step 3: Product Selection

Priya compares different smartphones by checking:

Price

Specifications

Customer ratings

Reviews

Seller information

After comparing products, she selects one smartphone.

Tables Used:

Products

Reviews

Sellers

Step 4: Add to Cart

Priya adds the selected smartphone to her shopping cart and updates the quantity to one.

Tables Used:

Cart

Cart_Items

Products

CASE STUDY

Step 5: Place Order

Priya proceeds to checkout, selects her delivery address, and confirms the order.
The system creates a new order and records all selected products.

Tables Used:

Orders

Order_Items

Addresses

Step 6: Payment

Priya pays using UPI.

The payment is processed successfully, and the payment status is updated.

Tables Used:

Payments

Orders

Step 7: Shipping

The seller prepares the package, and the courier company ships it to Priya's address.
She receives tracking updates until the product is delivered.

Tables Used:

Shipments

Orders

Addresses

Step 8: Review and Rating

After receiving the smartphone, Priya gives the product a 5-star rating and writes a positive review.

This review becomes visible to future customers.

Tables Used:

Reviews

Customers

Products

CASE STUDY

Case Study Summary

This case study demonstrates how multiple database tables work together to support a complete online shopping experience. Every customer action—from logging in and searching for a product to placing an order, making a payment, tracking the shipment, and submitting a review—is stored and managed through a well-designed relational database. Such a database ensures data accuracy, consistency, and efficient business operations.

Database Requirement

Database Requirement Analysis

Introduction

A relational database is the foundation of an e-commerce platform like Amazon. Every customer activity, such as registering an account, searching for products, placing an order, making a payment, and writing a review, generates data. To manage this data efficiently, the database must be carefully designed with appropriate entities, attributes, relationships, and constraints.

The following entities have been identified after analyzing Amazon's business processes.

1. Customers

Purpose: Stores information about customers who use the platform to purchase products.

Primary Key:

customer_id

Main Attributes:

first_name

last_name

email

phone_number

password

date_of_birth

gender

account_status

Created_at

2. Sellers

Purpose: Stores information about businesses or individuals who sell products on Amazon.

Primary Key:

seller_id

Main Attributes:

seller_name

business_name

email

phone_number

gst_number

rating

created_at

3. Categories

Purpose: Groups products into different categories such as Electronics, Clothing, Books, and Home Appliances.

Primary Key:

Category_id

Database Requirement Analysis

Main Attributes:

category_name

Description

4. Products

Purpose: Stores details of all products available for sale.

Primary Key:

product_id

Foreign Keys:

seller_id

category_id

Main Attributes:

product_name

brand

description

price

discount

Stock_quantity

5. Addresses

Purpose: Stores customer delivery addresses.

Primary Key:

address_id

Foreign Key:

customer_id

6. Cart

Purpose: Represents the shopping cart for each customer.

Primary Key:

cart_id

Foreign Key:

Customer_id

7. Cart_Items

Purpose: Stores products added to a shopping cart.

Primary Key:

cart_item_id

Foreign Keys:

cart_id

Product_id

8. Orders

Purpose: Stores order details placed by customers.

Primary Key:

order_id

Foreign Keys:

customer_id

Address_id

Database Requirement Analysis

9. Order_Items

Purpose: Stores products included in each order.

Primary Key:

order_item_id

Foreign Keys:

order_id

Product_id

10. Payments

Purpose: Stores payment details for customer orders.

Primary Key:

payment_id

Foreign Key:

Order_id

11. Shipments

Purpose: Tracks the delivery status of orders.

Primary Key:

shipment_id

Foreign Key:

Order_id

12. Reviews

Purpose: Stores customer ratings and reviews for purchased products.

Primary Key:

review_id

Foreign Keys:

customer_id

Product_id

13. Wishlist

Purpose: Stores products saved by customers for future purchases.

Primary Key:

wishlist_id

Foreign Keys:

customer_id

Product_id

14. Inventory

Purpose: Maintains product stock levels and availability.

Primary Key:

inventory_id

Foreign Key:

Database Requirement Analysis

product_id

Conclusion

The identified entities represent the core operations of an Amazon-like e-commerce platform. Each entity stores a specific type of information and is connected to other entities through primary and foreign keys. This structure minimizes data redundancy, maintains data integrity, and supports efficient order processing, inventory management, and business reporting.

Database Schema Description

DATABASE SCHEMA DESCRIPTION

Database Schema Description

This database is designed to manage the operations of an Amazon-like e-commerce platform. It stores information about customers, sellers, products, orders, payments, shipments, reviews, wishlists, inventory, and gifts. The schema uses primary keys and foreign keys to maintain relationships and ensure data integrity.

Tables Description

Customers – Stores customer details such as name, email, phone number, and account information.

Address – Stores customer shipping and billing addresses. Each address belongs to a customer.

Sellers – Stores information about sellers who list products on the platform.

Categories – Stores product categories such as Electronics, Clothing, Books, Home & Kitchen, etc.

Products – Stores product information including seller, category, price, discount, and stock quantity.

Cart – Represents a shopping cart for each customer.

Cart_Items – Stores the products added to a customer's cart.

Orders – Stores customer order information along with the shipping address.

Order_Items – Stores individual products included in each order.

Payments – Stores payment details for orders, including the payment method.

Shipments – Stores shipment details such as tracking number, courier, shipment status, shipment date, expected delivery date, and delivery date.

Review – Stores customer ratings and reviews for purchased products.

Wishlist – Stores products that customers save for future purchase.

Inventory – Tracks products available in stock.

Gifts – Stores gift order details including gift message, gift wrapping option, and recipient name.

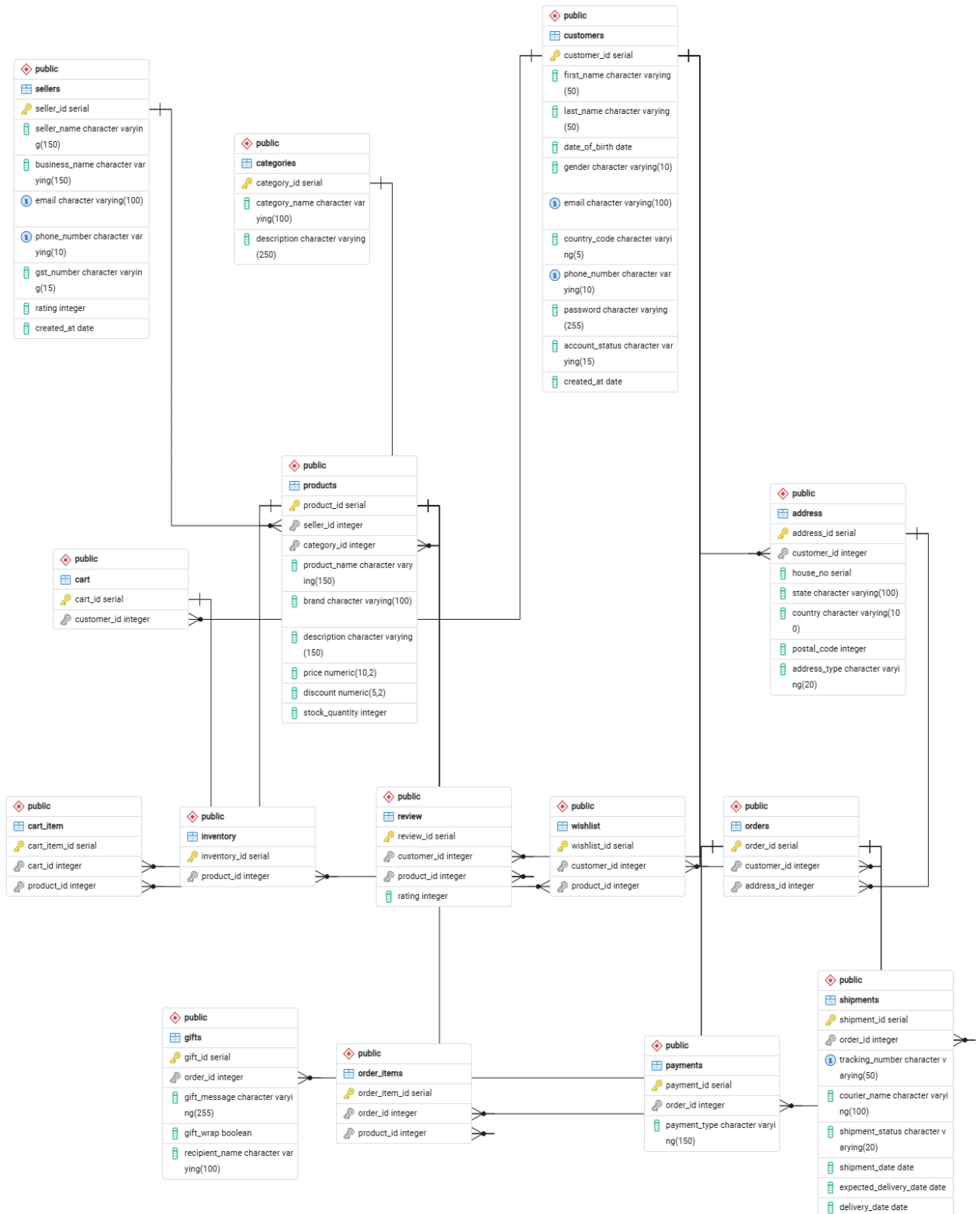
DATABASE SCHEMA DESCRIPTION

Relationships

1. One customer can have multiple addresses.
2. One seller can sell many products.
3. One category can contain many products.
4. One customer can place many orders.
5. One order can contain multiple products through the Order_Items table.
6. Each order has one payment record.
7. Each order has one shipment record.
8. Customers can review products after purchase.
9. Customers can add products to their wishlist and shopping cart.

ERD DIAGRAM

ERD DIAGRAM



SCHEMA TABLE

SCHEMA TABLE

```
CREATE DATABASE ecommerce_amazon_db;
```

```
--Create Table|
```

```
CREATE TABLE Customers(  
customer_id SERIAL PRIMARY KEY,  
first_name VARCHAR(50) NOT NULL,  
last_name VARCHAR(50) NOT NULL,  
date_of_birth DATE NOT NULL,  
gender VARCHAR(10) CHECK(gender IN('Female','Male','Other','Prefer not to say')),  
email VARCHAR(100) UNIQUE NOT NULL,  
country_code VARCHAR(5) NOT NULL,  
phone_number VARCHAR(10) UNIQUE NOT NULL ,  
password VARCHAR(255) NOT NULL,  
account_status VARCHAR(15) CHECK(account_status IN ('active')),  
created_at DATE DEFAULT(CURRENT_DATE)  
);
```

```
CREATE TABLE Address(  
address_id SERIAL PRIMARY KEY,  
customer_id INT ,FOREIGN KEY(customer_id) REFERENCES Customers(customer_id),  
house_no SERIAL NOT NULL,  
state VARCHAR(100) NOT NULL,  
country VARCHAR(100) NOT NULL,  
postal_code INT NOT NULL,  
address_type VARCHAR(20) CHECK(address_type IN('Home','Office','Other'))  
);
```

```
CREATE TABLE Sellers(  
seller_id SERIAL PRIMARY KEY,  
seller_name VARCHAR(150) NOT NULL,  
business_name VARCHAR(150) NOT NULL,  
email VARCHAR(100) NOT NULL u  
phone_number VARCHAR(10) UNIQUE NOT NULL,  
gst_number VARCHAR(15) NOT NULL,  
rating INT CHECK(rating<=5),  
created_at DATE DEFAULT(CURRENT_DATE)
```

```

CREATE TABLE Categories(
  category_id SERIAL PRIMARY KEY,
  category_name VARCHAR(100) NOT NULL,
  description VARCHAR(250) NOT NULL
);
CREATE TABLE Products(
  product_id SERIAL PRIMARY KEY,
  seller_id INT ,FOREIGN KEY(seller_id) REFERENCES Sellers(seller_id),
  category_id INT,FOREIGN KEY(category_id) REFERENCES Categories(category_id),
  product_name VARCHAR(150) NOT NULL,
  brand VARCHAR(100) NOT NULL,
  description VARCHAR(150) NOT NULL,
  price DECIMAL(5,2) CHECK(price>0),
  discount DECIMAL(5,2) CHECK(discount>10),
  stock_quantity INT CHECK(stock_quantity>0)
);
CREATE TABLE Cart(
  cart_id SERIAL PRIMARY KEY,
  customer_id INT,FOREIGN KEY(customer_id) REFERENCES customers(customer_id)
);
CREATE TABLE Cart_item(
  cart_item_id SERIAL PRIMARY KEY,
  cart_id INT,FOREIGN KEY(cart_id) REFERENCES Cart(cart_id),
  product_id INT,FOREIGN KEY(product_id) REFERENCES Products(product_id)
);
CREATE TABLE Orders(
  order_id SERIAL PRIMARY KEY,
  customer_id INT,FOREIGN KEY(customer_id) REFERENCES Customers(customer_id),
  address_id INT,FOREIGN KEY(address_id) REFERENCES Address(address_id)
);
CREATE TABLE Order_items(
  order_item_id SERIAL PRIMARY KEY,
  order_id INT ,FOREIGN KEY(order_id) REFERENCES Orders(order_id),
  product_id INT ,FOREIGN KEY(product_id) REFERENCES Products(product_id)
  product_id INT ,FOREIGN KEY(product_id) REFERENCES Products(product_id)
);
CREATE TABLE Payments(
  payment_id SERIAL PRIMARY KEY,
  order_id INT,FOREIGN KEY(order_id) REFERENCES Orders(order_id),
  payment_type VARCHAR(150) NOT NULL CHECK(payment_type IN('cash','UPI','card'))
);
CREATE TABLE Shipments(
  shipment_id SERIAL PRIMARY KEY,
  order_id INT,FOREIGN KEY(order_id) REFERENCES Orders(order_id),
  tracking_number VARCHAR(50) UNIQUE,
  courier_name VARCHAR(100) NOT NULL,
  shipment_status VARCHAR(20) CHECK(shipment_status IN('Pending','Shipment','Out for','Delivery','Delivered','Cancelled')),
  shipment_date DATE ,
  expected_delivery_date DATE,
  delivery_date DATE
);
CREATE TABLE Review(
  review_id SERIAL PRIMARY KEY,
  customer_id INT ,FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
  product_id INT ,FOREIGN KEY (product_id) REFERENCES Products(product_id),
  rating INT CHECK(rating<=5)
);
CREATE TABLE Wishlist(
  wishlist_id SERIAL PRIMARY KEY,
  customer_id INT ,FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),
  product_id INT ,FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
CREATE TABLE Inventory(
  inventory_id SERIAL PRIMARY KEY,
  product_id INT ,FOREIGN KEY (product_id) REFERENCES Products(product_id)
);

```

```
);  
  
CREATE TABLE Inventory(  
inventory_id SERIAL PRIMARY KEY,  
product_id INT ,FOREIGN KEY (product_id) REFERENCES Products(product_id)  
);
```

```
CREATE TABLE Gifts(  
gift_id SERIAL PRIMARY KEY,  
order_id INT,FOREIGN KEY(order_id) REFERENCES Orders(order_id),  
gift_message VARCHAR(255) ,  
gift_wrap BOOLEAN DEFAULT FALSE,  
recipient_name VARCHAR(100) NOT NULL  
);
```

```
ALTER TABLE order_items ADD COLUMN quantity INT DEFAULT 1,  
ADD COLUMN sellinh_price DECIMAL(10,2),  
ADD COLUMN discount DECIMAL(5,2) DEFAULT 0;  
ALTER TABLE order_items RENAME COLUMN sellinh_price TO selling_price;
```

```
UPDATE order_items oi SET selling_price=p.price  
FROM products p WHERE oi.product_id=p.product_id;  
SELECT* FROM order_items;
```

INSERT TABLE

INSERT TABLE

```
--Insert the Data
INSERT INTO Customers
(first_name,last_name,date_of_birth,gender,email,country_code,phone_number,password,account_status)
VALUES
('Rahul','Sharma','1998-05-12','Male','rahul.sharma@gmail.com','+91','9876543210','Rahul@123','active'),
('Priya','Patel','1997-08-20','Female','priya.patel@gmail.com','+91','9876543211','Priya@123','active'),
('Amit','Verma','1995-03-18','Male','amit.verma@gmail.com','+91','9876543212','Amit@123','active'),
('Neha','Gupta','1999-11-02','Female','neha.gupta@gmail.com','+91','9876543213','Neha@123','active'),
('Rohit','Singh','1996-06-10','Male','rohit.singh@gmail.com','+91','9876543214','Rohit@123','active'),
('Sneha','Joshi','1998-01-09','Female','sneha.joshi@gmail.com','+91','9876543215','Sneha@123','active'),
('Arjun','Kumar','1994-12-15','Male','arjun.kumar@gmail.com','+91','9876543216','Arjun@123','active'),
('Pooja','Yadav','1997-09-28','Female','pooja.yadav@gmail.com','+91','9876543217','Pooja@123','active'),
('Vikas','Mehta','1995-05-19','Male','vikas.mehta@gmail.com','+91','9876543218','Vikas@123','active'),
('Anjali','Roy','1998-04-23','Female','anjali.roy@gmail.com','+91','9876543219','Anjali@123','active'),
('Karan','Malhotra','1996-07-14','Male','karan.m@gmail.com','+91','9876543220','Karan@123','active'),
('Meera','Nair','1999-10-30','Female','meera.nair@gmail.com','+91','9876543221','Meera@123','active'),
('Suresh','Reddy','1993-01-18','Male','suresh.reddy@gmail.com','+91','9876543222','Suresh@123','active'),
('Kavita','Shah','1998-08-16','Female','kavita.shah@gmail.com','+91','9876543223','Kavita@123','active'),
('Manish','Agarwal','1995-11-11','Male','manish.agarwal@gmail.com','+91','9876543224','Manish@123','active'),
('Nisha','Kapoor','1997-12-09','Female','nisha.kapoor@gmail.com','+91','9876543225','Nisha@123','active'),
('Deepak','Soni','1994-04-01','Male','deepak.soni@gmail.com','+91','9876543226','Deepak@123','active'),
('Riya','Bose','1998-06-21','Female','riya.bose@gmail.com','+91','9876543227','Riya@123','active'),
('Ajay','Mishra','1996-02-18','Male','ajay.mishra@gmail.com','+91','9876543228','Ajay@123','active'),
('Komal','Arora','1999-09-05','Female','komal.arora@gmail.com','+91','9876543229','Komal@123','active'),
('Akash','Chauhan','1997-03-15','Male','akash.chauhan@gmail.com','+91','9876543230','Akash@123','active'),
('Divya','Sharma','1998-08-27','Female','divya.sharma@gmail.com','+91','9876543231','Divya@123','active'),
('Nitin','Pandey','1995-01-19','Male','nitin.pandey@gmail.com','+91','9876543232','Nitin@123','active'),
('Swati','Jain','1999-05-11','Female','swati.jain@gmail.com','+91','9876543233','Swati@123','active'),
('Harsh','Kapoor','1996-12-22','Male','harsh.kapoor@gmail.com','+91','9876543234','Harsh@123','active'),
('Isha','Malik','1997-07-04','Female','isha.malik@gmail.com','+91','9876543235','Isha@123','active'),
('Mohit','Bansal','1994-09-14','Male','mohit.bansal@gmail.com','+91','9876543236','Mohit@123','active'),
('Sakshi','Goyal','1998-02-09','Female','sakshi.goyal@gmail.com','+91','9876543237','Sakshi@123','active'),
('Yash','Khanna','1995-11-30','Male','yash.khanna@gmail.com','+91','9876543238','Yash@123','active'),
```

```
INSERT INTO Address
(customer_id, house_no, state, country, postal_code, address_type)
VALUES
(1,101,'Delhi','India',110001,'Home'),
(2,102,'Maharashtra','India',400001,'Home'),
(3,103,'Karnataka','India',560001,'Office'),
(4,104,'Tamil Nadu','India',600001,'Home'),
(5,105,'Telangana','India',500001,'Home'),
(6,106,'Gujarat','India',380001,'Office'),
(7,107,'Punjab','India',141001,'Home'),
(8,108,'Rajasthan','India',302001,'Home'),
(9,109,'Uttar Pradesh','India',226001,'Home'),
(10,110,'Kerala','India',682001,'Office'),

(11,111,'Delhi','India',110002,'Home'),
(12,112,'Maharashtra','India',400002,'Home'),
(13,113,'Karnataka','India',560002,'Office'),
(14,114,'Tamil Nadu','India',600002,'Home'),
(15,115,'Telangana','India',500002,'Home'),
(16,116,'Gujarat','India',380002,'Office'),
(17,117,'Punjab','India',141002,'Home'),
(18,118,'Rajasthan','India',302002,'Home'),
(19,119,'Uttar Pradesh','India',226002,'Home'),
(20,120,'Kerala','India',682002,'Office'),

(21,121,'Delhi','India',110003,'Home'),
(22,122,'Maharashtra','India',400003,'Home'),
(23,123,'Karnataka','India',560003,'Office'),
(24,124,'Tamil Nadu','India',600003,'Home'),
(25,125,'Telangana','India',500003,'Home'),
(26,126,'Gujarat','India',380003,'Office'),
(27,127,'Punjab','India',141003,'Home'),
(28,128,'Rajasthan','India',302003,'Home'),
(29,129,'Uttar Pradesh','India',226003,'Home').
```

INSERT INTO Sellers

(seller_name, business_name, email, phone_number, gst_number, rating)

VALUES

```
('Amit Sharma','Sharma Electronics','amit.sharma@shop.com','9000000001','27ABCDE1234F1Z5',5),
('Priya Patel','Patel Fashion','priya.patel@shop.com','9000000002','24ABCDE1234F1Z6',4),
('Rahul Verma','Verma Mobiles','rahul.verma@shop.com','9000000003','09ABCDE1234F1Z7',5),
('Neha Gupta','Gupta Home Decor','neha.gupta@shop.com','9000000004','07ABCDE1234F1Z8',4),
('Rohit Singh','Singh Sports','rohit.singh@shop.com','9000000005','10ABCDE1234F1Z9',5),
('Pooja Mehta','Mehta Books','pooja.mehta@shop.com','9000000006','29ABCDE1234F1Z1',4),
('Vikas Kumar','Kumar Grocery','vikas.kumar@shop.com','9000000007','06ABCDE1234F1Z2',3),
('Sneha Joshi','Joshi Beauty','sneha.joshi@shop.com','9000000008','08ABCDE1234F1Z3',5),
('Arjun Nair','Nair Furniture','arjun.nair@shop.com','9000000009','32ABCDE1234F1Z4',4),
('Anjali Roy','Roy Kitchen','anjali.roy@shop.com','9000000010','19ABCDE1234F1Z5',5),
('Deepak Shah','Shah Electronics','deepak@shah.com','9000000016','27ABCDE1234F2Z2',4),
('Komal Arora','Fashion Point','komal@fashion.com','9000000017','27ABCDE1234F2Z3',5),
('Nitin Bansal','Mobile Planet','nitin@mobile.com','9000000018','27ABCDE1234F2Z4',4),
('Aakash Mishra','Computer Store','aakash@pc.com','9000000019','27ABCDE1234F3Z5',5),
('Harsh Gupta','Gaming Hub','harsh@gaming.com','9000000020','27ABCDE1234F3Z6',5),
('Simran Kaur','Ladies Collection','simran@ladies.com','9000000021','27ABCDE1234F3Z7',4),
('Tarun Malhotra','Malhotra Mart','tarun@mart.com','9000000022','27ABCDE1234F3Z8',5),
('Rakesh Soni','Jewellery House','rakesh@jewel.com','9000000023','27ABCDE1234F3Z9',4),
('Mohit Arora','Laptop World','mohit@laptop.com','9000000024','27ABCDE1234F3Z1',5),
('Kavita Rao','Furniture Point','kavita@furniture.com','9000000025','27ABCDE1234F3Z2',5),
('Ashish Garg','Garg Traders','ashish@garg.com','9000000026','27ABCDE1234F3Z3',4),
('Divya Sharma','Toy Kingdom','divya@toys.com','9000000027','27ABCDE1234F3Z4',5),
('Lokesh Jain','Stationery Store','lokesh@stationery.com','9000000028','27ABCDE1234F4Z5',5),
('Bhavna Singh','Gift Gallery','bhavna@gifts.com','9000000029','27ABCDE1234F4Z6',4),
('Ravi Choudhary','Daily Needs','ravi@daily.com','9000000030','27ABCDE1234F4Z7',5),
('Nisha Gupta','Kitchen Mart','nisha@kitchen.com','9000000031','27ABCDE1234F4Z8',5),
('Abhishek Singh','Phone Store','abhishek@phone.com','9000000032','27ABCDE1234F4Z9',4),
('Preeti Sharma','Beauty Palace','preeti@beauty.com','9000000033','27ABCDE1234F4Z1',5),
('Hemant Verma','Sports Arena','hemant@sports.com','9000000034','27ABCDE1234F4Z2',4),
('Jyoti Patel','Home Essentials','jyoti@home.com','9000000035','27ABCDE1234F4Z3',5),
```

```
INSERT INTO Cart (customer_id)
VALUES
(1),(2),(3),(4),(5),(6),
(7),(8),(9),(10),(11),
(12),(13),(14),(15),(16),
(17),(18),(19),(20),(21),
(22),(23),(24),(25),(26),
(27),(28),(29),(30),(31),
(32),(33),(34),(35),(36),
(37),(38),(39),(40),(41),
(42),(43),(44),(45),(46),
(47),(48),(49),(50);
INSERT INTO Cart_item(cart_id, product_id)
VALUES
(1,1),(2,2),(3,3),(4,4),(5,5),
(6,6),(7,7),(8,8),(9,9),(10,10),
(11,11),(12,12),(13,13),(14,14),(15,15),
(16,16),(17,17),(18,18),(19,19),(20,20),
(21,21),(22,22),(23,23),(24,24),(25,25),
(26,26),(27,27),(28,28),(29,29),(30,30),
(31,31),(32,32),(33,33),(34,34),(35,35),
(36,36),(37,37),(38,38),(39,39),(40,40),
(41,41),(42,42),(43,43),(44,44),(45,45),
(46,46),(47,47),(48,48),(49,49),(50,50);
INSERT INTO Orders
(customer_id, address_id)
VALUES
(1,1),
(2,2),
(3,3),
(4,4),
(5,5),
(6,6),
(7,7),
```

```
INSERT INTO Order_Items(order_id, product_id)
VALUES
```

```
(1,1),(2,2),(3,3),(4,4),(5,5),
(6,6),(7,7),(8,8),(9,9),(10,10),
(11,11),(12,12),(13,13),(14,14),(15,15),
(16,16),(17,17),(18,18),(19,19),(20,20),
(21,21),(22,22),(23,23),(24,24),(25,25),
(26,26),(27,27),(28,28),(29,29),(30,30),
(31,31),(32,32),(33,33),(34,34),(35,35),
(36,36),(37,37),(38,38),(39,39),(40,40),
(41,41),(42,42),(43,43),(44,44),(45,45),
(46,46),(47,47),(48,48),(49,49),(50,50);
```

```
INSERT INTO Payments(order_id,payment_type)
VALUES
```

```
(1, 'UPI'),
(2, 'card'),
(3, 'cash'),
(4, 'UPI'),
(5, 'card'),
(6, 'cash'),
(7, 'UPI'),
(8, 'card'),
(9, 'cash'),
(10, 'UPI'),
(11, 'card'),
(12, 'cash'),
(13, 'UPI'),
(14, 'card'),
(15, 'cash'),
(16, 'UPI'),
(17, 'card'),
(18, 'cash'),
(19, 'UPI'),
(20, 'card'),
```



```

INSERT INTO Shipments
(order_id,tracking_number,courier_name,shipment_status,shipment_date,expected_delivery_date,delivery_date)
VALUES
(1,'TRK1001','Delhivery','Delivered','2025-07-01','2025-07-05','2025-07-04'),
(2,'TRK1002','Blue Dart','Delivered','2025-07-02','2025-07-06','2025-07-05'),
(3,'TRK1003','DTDC','Shipment','2025-07-03','2025-07-07',NULL),
(4,'TRK1004','Ecom Express','Pending','2025-07-04','2025-07-08',NULL),
(5,'TRK1005','India Post','Out for','2025-07-05','2025-07-09',NULL),
(6,'TRK1006','Delhivery','Delivered','2025-07-06','2025-07-10','2025-07-09'),
(7,'TRK1007','Blue Dart','Delivered','2025-07-07','2025-07-11','2025-07-10'),
(8,'TRK1008','DTDC','Shipment','2025-07-08','2025-07-12',NULL),
(9,'TRK1009','Ecom Express','Pending','2025-07-09','2025-07-13',NULL),
(10,'TRK1010','India Post','Delivered','2025-07-10','2025-07-14','2025-07-13'),
(11,'TRK1011','Delhivery','Delivered','2025-07-11','2025-07-15','2025-07-14'),
(12,'TRK1012','Blue Dart','Shipment','2025-07-12','2025-07-16',NULL),
(13,'TRK1013','DTDC','Pending','2025-07-13','2025-07-17',NULL),
(14,'TRK1014','Ecom Express','Out for','2025-07-14','2025-07-18',NULL),
(15,'TRK1015','India Post','Delivered','2025-07-15','2025-07-19','2025-07-18'),
(16,'TRK1016','Delhivery','Delivered','2025-07-16','2025-07-20','2025-07-19'),
(17,'TRK1017','Blue Dart','Shipment','2025-07-17','2025-07-21',NULL),
(18,'TRK1018','DTDC','Pending','2025-07-18','2025-07-22',NULL),
(19,'TRK1019','Ecom Express','Out for','2025-07-19','2025-07-23',NULL),
(20,'TRK1020','India Post','Delivered','2025-07-20','2025-07-24','2025-07-23'),
(21,'TRK1021','Delhivery','Delivered','2025-07-21','2025-07-25','2025-07-24'),
(22,'TRK1022','Blue Dart','Shipment','2025-07-22','2025-07-26',NULL),
(23,'TRK1023','DTDC','Pending','2025-07-23','2025-07-27',NULL),
(24,'TRK1024','Ecom Express','Out for','2025-07-24','2025-07-28',NULL),
(25,'TRK1025','India Post','Delivered','2025-07-25','2025-07-29','2025-07-28'),
(26,'TRK1026','Delhivery','Delivered','2025-07-26','2025-07-30','2025-07-29'),
(27,'TRK1027','Blue Dart','Shipment','2025-07-27','2025-07-31',NULL),
(28,'TRK1028','DTDC','Pending','2025-07-28','2025-08-01',NULL),
(29,'TRK1029','Ecom Express','Out for','2025-07-29','2025-08-02',NULL),
(30,'TRK1030','India Post','Delivered','2025-07-30','2025-08-03','2025-08-02'),
(31,'TRK1031','Delhivery','Delivered','2025-07-31','2025-08-04','2025-08-03').

```

```
INSERT INTO Review
(customer_id,product_id,rating)
VALUES
(1,1,1),
(2,2,4),
(3,3,5),
(4,4,3),
(5,5,4),
(6,6,5),
(7,7,4),
(8,8,5),
(9,9,3),
(10,10,4),
(11,11,5),
(12,12,4),
(13,13,5),
(14,14,3),
(15,15,4),
(16,16,5),
(17,17,4),
(18,18,5),
(19,19,3),
(20,20,4),
(21,21,5),
(22,22,4),
(23,23,5),
(24,24,3),
(25,25,4),
(26,26,5),
(27,27,4),
(28,28,5),
(29,29,3),
(30,30,4),
(31,31,5),
```

```
INSERT INTO Wishlist (customer_id, product_id) VALUES
```

```
(1,5),(2,8),(3,12),(4,15),(5,20),  
(6,25),(7,30),(8,35),(9,40),(10,45),  
(11,2),(12,4),(13,6),(14,8),(15,10),  
(16,12),(17,14),(18,16),(19,18),(20,20),  
(21,22),(22,24),(23,26),(24,28),(25,30),  
(26,32),(27,34),(28,36),(29,38),(30,40),  
(31,42),(32,44),(33,46),(34,48),(35,50),  
(36,1),(37,3),(38,5),(39,7),(40,9),  
(41,11),(42,13),(43,15),(44,17),(45,19),  
(46,21),(47,23),(48,25),(49,27),(50,29);
```

```
INSERT INTO Inventory (product_id) VALUES
```

```
(1),(2),(3),(4),(5),  
(6),(7),(8),(9),(10),  
(11),(12),(13),(14),(15),  
(16),(17),(18),(19),(20),  
(21),(22),(23),(24),(25),  
(26),(27),(28),(29),(30),  
(31),(32),(33),(34),(35),  
(36),(37),(38),(39),(40),  
(41),(42),(43),(44),(45),  
(46),(47),(48),(49),(50);
```

```
INSERT INTO Gifts (order_id, gift_message, gift_wrap, recipient_name) VALUES
```

```
(1,'Happy Birthday',TRUE,'Aarav'),  
(2,'Best Wishes',FALSE,'Ananya'),  
(3,'Congratulations',TRUE,'Rahul'),  
(4,'Happy Anniversary',TRUE,'Priya'),  
(5,'Enjoy Your Gift',FALSE,'Rohan'),  
(6,'All the Best',TRUE,'Sneha'),  
(7,'Happy Birthday',FALSE,'Karan'),  
(8,'Congratulations',TRUE,'Neha'),  
(9,'With Love',TRUE,'Aditi'),  
(10,'Happy Moments',FALSE,'Arjun'),  
(11,'Best Wishes',TRUE,'Riva')
```

SQL QUERY

SQL QUERY

```
-- QUERIES
--VIEW OF TABLE
SELECT * FROM Customers;
SELECT * FROM address;
SELECT * FROM cart_item;
SELECT * FROM cart;
SELECT * FROM cart_items;
SELECT * FROM catgories;
SELECT * FROM orders;
SELECT * FROM order_items;
SELECT * FROM payments;
SELECT * FROM products;
SELECT * FROM sellers;
SELECT * FROM shipments;
SELECT * FROM review;
SELECT * FROM wishlist;
SELECT * FROM inventory;
SELECT * FROM gifts;

--Show products with price greater then 5000
SELECT * FROM products WHERE discount >15;
-- Show products ordered by price
SELECT * FROM products ORDER BY price DESC;
-- Count total customers
SELECT DISTINCT COUNT(*) AS total_customer FROM customers;
-- Count total products
SELECT DISTINCT COUNT(*) AS total_product FROM products;
--Show the distinct brands
SELECT DISTINCT brand FROM products;
-- Average product price
SELECT ROUND(AVG(price),2) AS averge_price FROM products;
-- . . . . .

-- Products with highest stock
SELECT * FROM products ORDER BY stock_quantity DESC ;
--Show all delivered shipments
SELECT * FROM shipments WHERE shipment_status ='Delivered';
-- Show all reviews with 5-star rating
SELECT * FROM review WHERE rating=5;
-- Total inventory
SELECT COUNT(*) AS total_inventory FROM inventory;
-- Average rating
SELECT AVG(rating) AS avg_rating FROM review ;
-- Number of products per seller
SELECT seller_id,COUNT(*) AS no_product FROM products GROUP BY seller_id;
-- Number of shipments by courier
SELECT courier_name ,COUNT(shipment_id) AS total_shipment FROM shipments GROUP BY courier_name;
--Number of wishlist items
SELECT COUNT(wishlist_id) AS total_wishlist FROM wishlist;
-- Number of reviews
SELECT COUNT(review_id) AS total_review FROM review;

-- JOIN FUNCTIONS
--Customer with address
SELECT s.first_name,s.last_name,e.house_no,e.state,e.country,e.postal_code,e.address_type FROM customers s JOIN address e ON
s.customer_id=e.customer_id;
--Customer with orders
SELECT s.first_name,s.last_name,o.order_id FROM customers s JOIN orders o ON s.customer_id=o.customer_id;
--Customer with payment details
SELECT s.first_name,s.last_name,o.order_id,p.payment_id,p.payment_type FROM customers s JOIN orders o ON s.customer_id=o.customer_id
JOIN payments p ON o.order_id=p.order_id;
-- Orders with products
SELECT o.order_id ,p.product_id,p.product_name FROM order_items o JOIN products p ON o.product_id=p.product_id;
-- Gifts with orders
SELECT o.order_id,g.gift_id,g.gift_message,g.gift_wrap FROM orders o JOIN gifts g ON o.order_id=g.order_id;
-- Shipment with courier
SELECT courier_name ,COUNT(shipment_id)AS total_shipment FROM shipments GROUP BY courier_name;
```

```

-- VIEW QUERY
--Customer completet purchase history
SELECT c.customer_id,c.first_name||' '||c.last_name as full_name,o.order_id,p.product_name,
cat.category_name,s.seller_name,
p.price,pay.payment_type,sh.tracking_number,sh.courier_name,
sh.shipment_status,sh.shipment_date,sh.expected_delivery_date,a.state, a.country
FROM Customers c
JOIN Orders o
ON c.customer_id = o.customer_id
JOIN Order_Items oi
ON o.order_id = oi.order_id
JOIN Products p
ON oi.product_id = p.product_id
JOIN Categories cat
ON p.category_id = cat.category_id
JOIN Sellers s
ON p.seller_id = s.seller_id
JOIN Payments pay
ON o.order_id = pay.order_id
JOIN Shipments sh
ON o.order_id = sh.order_id
JOIN Address a
ON o.address_id = a.address_id
ORDER BY c.customer_id, o.order_id;

-- Order details with payment and shipment
SELECT o.order_id,p.payment_type,s.tracking_number,s.courier_name,s.shipment_date,s.delivery_date FROM orders o JOIN payments p ON
o.order_id=p.order_id JOIN shipments s ON p.order_id=s.order_id;

SELECT product_name,price FROM products WHERE price=(SELECT MAX(price) FROM products);
--Products above average price
SELECT * FROM products WHERE price>(SELECT AVG(price) FROM products);
--Cusomers without orders
SELECT * FROM customers c LEFT JOIN orders o ON c.customer_id=o.customer_id WHERE o.order_id IS NULL;
--Order without shipment
SELECT * FROM orders o LEFT JOIN shipments s ON o.order_id=s.order_id WHERE s.shipment_id IS NULL;
-- Orders without payment
SELECT * FROM orders o LEFT JOIN payments p ON o.order_id=p.order_id WHERE p.payment_id IS NULL;
--Products in more than one wishlist
SELECT p.product_name,COUNT(w.wishlist_id)AS total_wishlist FROM products p JOIN wishlist w ON
p.product_i
--Categories with more than five products
SELECT c.category_name,COUNT(*)AS total_products FROM categories c JOIN products p ON c.category_id=p.category_id
GROUP BY c.category_name HAVING COUNT(*)>5;
-- Customers with more than one order
SELECT c.customer_id,c.first_name,c.last_name,COUNT(*)AS total_order FROM customers c JOIN orders o
ON c.customer_id=o.customer_id GROUP BY c.customer_id,c.first_name,c.last_name HAVING COUNT(*)>1;

-- SUB QUERIES
-- Products with maximum discount
SELECT * FROM products WHERE discount=(SELECT MAX(discount) FROM products);

-- SECOND MOST PRODUCT EXPENSIVE PRICE
SELECT MAX(price) FROM products WHERE price<(SELECT MAX(price) FROM products);

-- CUSTOMER WHO PLACED ORDERS
SELECT * FROM customers WHERE customer_id IN (SELECT customer_id FROM orders);

-- PRODOCTS NEVER ORDER
SELECT * FROM products WHERE product_id NOT IN (SELECT product_id FROM order_items);
-- PRODUCTS ABOVE THE AVERAGE PRICE
SELECT * FROM products WHERE price>(SELECT AVG(price) FROM products);

```

```

-- WINDOW FUNTIONS
-- Top 5 expensive products
SELECT * FROM products ORDER BY price DESC LIMIT 5;
SELECT * FROM (SELECT product_id,product_name,price,DENSE_RANK()OVER(ORDER BY price DESC)AS
rnk FROM products ) as t WHERE rnk<5;
--Top customers by number of orders
SELECT c.customer_id,c.first_name,c.last_name,
COUNT(o.order_id) as total_order,
RANK()OVER(ORDER BY COUNT(o.order_id) DESC ) AS rnk
FROM customers c JOIN orders o
ON c.customer_id=o.customer_id GROUP BY c.customer_id,c.first_name,c.last_name;
--Product ranking
SELECT product_name,price,ROW_NUMBER()OVER(ORDER BY price DESC)AS rnK FROM products;
--Revenue ranking
SELECT order_id,SUM(quantity*selling_price) AS rev,DENSE_RANK()OVER(ORDER BY SUM(quantity*selling_price) DESC) AS RNK
FROM order_items GROUP BY order_id;
-- Dense rank products by price
SELECT product_name,price,DENSE_RANK()OVER(ORDER BY price DESC)AS rnk FROM products;
--Category ranking
SELECT c.category_name,SUM(p.price),RANK()OVER(ORDER BY SUM(p.price) DESC)AS rnk FROM products p JOIN
categories c ON p.category_id=c.category_id GROUP BY c.category_name;
--ROW_Number
SELECT product_name,discount,ROW_NUMBER()OVER(ORDER BY discount DESC)AS rnk FROM products;
--Rank
SELECT product_name,discount,RANK()OVER(ORDER BY discount DESC)AS rnk FROM products;
--Dense_Rank
SELECT product_name,discount,DENSE_RANK()OVER(ORDER BY discount DESC)AS rnk FROM products;
--Lead()
SELECT product_name,price,LEAD(price)OVER(ORDER BY price)AS next_price FROM products;
--lag()
SELECT product_name,price,LAG(price)OVER(ORDER BY price)AS previous_price FROM products;
--Running total

-- CTE QUERY

-- CUSTOMER ORDER COUNT
WITH customer_orders AS(
SELECT customer_id,COUNT(*) AS total_order
FROM orders GROUP BY customer_id)
SELECT c.customer_id,c.first_name,c.last_name,oi.total_order FROM
customers c JOIN customer_orders oi ON c.customer_id=oi.customer_id
ORDER BY oi.total_order DESC;

-- PRODUCTS PRICES ABOVE THE AVERAGE PRICE
WITH products_price AS(
SELECT AVG(price) AS avg_price
FROM products)
SELECT product_name,price FROM products
WHERE price>(SELECT avg_price FROM products_price);

```

BUSINESS REPORT ANALYSIS

BUSINESS REPORT ANALYSIS

```

--- BUSINESS REPORT

--- SOME QUESTION BUSINESS AND QUERY THEN INSIGHT OF BUSINESS
-- CUSTOMER ANALYSIS
-- HOW MANY CUSTOMER REGISTER?
SELECT COUNT(*) AS total_customer FROM customers;
-- 50 CUSTOMER ARE REGISTER

-- WHICH COUNTRY/STATE HAS MORE CUSTOMER REGISTER?
SELECT c.country,c.state,COUNT(*)AS total_customer FROM customers o JOIN address c
ON o.customer_id=c.customer_id GROUP BY c.country,c.state;
-- ONLY INDIA HAS MORE CUSOMTER AND ALL STATE WISE ALSO CUSTOMER IS SAME

-- WHO'S MOSTLY GENDER CUSOMTER ARE REGISTER?
SELECT gender,COUNT(*)AS TOTAL_CUSOMER FROM customers GROUP BY gender;
-- THE FEMALE AND MALE HAVE SAME CUSTOMER

-- WHICH CUSTOMER HAVE MORE ORDER?
SELECT c.customer_id,c.first_name,c.last_name,COUNT(*)AS TOTAL_ORDER FROM
customers c JOIN orders o ON c.customer_id=o.customer_id GROUP BY
c.customer_id,c.first_name,c.last_name ORDER BY TOTAL_ORDER DESC;
-- ALL MOST ALL CUSTOMER ORDER ONLY ONE TIMES

-- HOE MANY CUSTOMER GET REGISTERED BY MONTH?
SELECT TO_CHAR(created_at,'MONTH')AS month_name,COUNT(*) AS total_customer
FROM customers GROUP BY month_name;
-- JULY RECORD THE HIGHEST CUSTOMER REGISTRATIONS (50 CUSTOMERS)

-- PRODUCT ANALYSIS
-- HOW MANY PRODUCT?
SELECT COUNT(*) AS TOTAL_PRODUCT FROM products;
-- 51 PRODUCTS

--WHICH PRODUCTS HAVE MAX DISCOUNT ?
SELECT DISTINCT product_name,discout FROM products ORDER BY discount DESC LIMIT 1;
-- ONLY Boat Airdropes HAVE MAX DISCOUNT 25 %.

-- WHICH PRODUCT HAS GOING TO OUT OF STOCK?
SELECT DISTINCT product_name,stock_quantity FROM products ORDER BY stock_quantity ASC LIMIT 5;
-- THEY LIKE STOCK 15,25,30 HAS LOW STOCK

-- WHICH PRODUCT HAS MORE EXPENSIVE?
SELECT * FROM products ORDER BY price DESC LIMIT 10;
-- THEY ARE TOP 10 PRODUCT HAS MOST EXPENSIVE

-- WISHLIST_ANALYSIS
-- WHICH PRODUCTS ARE MOST POPULAR IN CUSTOMER WISHLST?
SELECT p.product_name ,COUNT(*)AS wishlist_count FROM products p JOIN wishlist w
ON p.product_id=w.product_id GROUP BY p.product_name HAVING COUNT(*)>1;
-- THE 8 PRODUCTS HAS MORE CUSTOEMR WISHLIST.

--- SHIPMENTS ANALYSIS
-- HOW MANY CUSTOMER ORDER ARE DELIVERED?
SELECT COUNT(*) FROM shipments WHERE shipment_status='Delivered';
-- 21 ORDER ARE DELVERED

-- WHICH COURIER ARE MOSTLY USED FOR SHIPMENTS?
SELECT courier_name,COUNT(*) shipment FROM shipments GROUP BY courier_name;
-- COURIER USSAGE IS EVENLY DISTRIBUTED AMONG AVAILBLE COURIER PARTNERS.

-- HOW MANY SHIPMENTS WERE DISPATCHED EACH MONTH?
SELECT TO_CHAR(shipment_date,'Month') AS month_name,EXTRACT(MONTH FROM shipment_date)AS month,
COUNT(*)AS total_shipment FROM shipments GROUP BY month_name,month ORDER BY month;
-- ONLY JULY HAS HIGHEST SHIPMENTS 31 THEN NEXT AUGEST

```

```

-- HOW MANY SHIPMENTS WERE DELIVERED EACH MONTH?
SELECT EXTRACT(MONTH FROM delivery_date) AS month,COUNT(*) AS delivery_order FROM shipments
WHERE shipment_status='Delivered' GROUP BY EXTRACT(MONTH FROM delivery_date) ORDER BY month;

-- PAYMENT ANALYSIS

-- WHICH PAYMENT TYPE ARE USED MORE ?
SELECT payment_type,COUNT(*) AS total_payment FROM payments GROUP BY payment_type;
-- THE UPI AND CARD HAVE 17 TIMES BY PAYMENT AND MORE PAYMENT TYPE USED IS CASH 18 TIMES .

-- INVENTORY ANALYSIS

-- WHICH PRODUCT HAS LOW STOCK?
SELECT product_name,stock_quantity FROM products WHERE stock_quantity <10;
-- THEY ARE NO PRODUCTS IS LOW STOCK

-- SELLER ANALYSIS

-- WHAT IS THE AVERAGE PRODUCT PRICE BY SELLER?
SELECT s.seller_name,s.business_name,ROUND(AVG(p.price),2) AS AVG_PRODUCT_BY_SELLER FROM sellers s JOIN products p
ON s.seller_id=p.seller_id GROUP BY s.seller_name,s.business_name ORDER BY AVG_PRODUCT_BY_SELLER DESC;
-- PRIYA PATEL HAS LARGEST AVG_PRODUCT_PRICE LIKE 41999.0

-- WHICH BUSINESS OR SELLER CREATED THE RECENT?
SELECT * FROM sellers WHERE created_at=CURRENT_DATE;
-- NO ONE SELLER ADDED

-- WHICH BUSINESS OR SELLER HIGHEST RATING?
SELECT * FROM sellers ORDER BY rating DESC ;
-- MAX RATING IS 5 CAMERE STORE HAS HIGHEST RATING

```

```

-- SALES ANALYSIS

-- TELL THE TOTAL SALES
SELECT SUM(quantity*selling_price) AS total_sales FROM order_items;
-- TOTAL SALES 6554090.0

-- HOW MANY PRODUCTS ARE SOLD?
SELECT SUM(quantity) AS total_sold_products FROM order_items;
-- 50 PRODUCTS ARE SOLD

-- TOP 5 PRODUCTS HAS HIGHEST REVENUE
SELECT p.product_name,SUM(o.quantity*o.selling_price)AS revenue FROM order_items o JOIN
products p ON o.product_id=p.product_id GROUP BY p.product_name ORDER BY revenue DESC LIMIT 5;
-- iPhone 15 HAS HIGHEST REVENUE 79999.0

-- WHICH CUSTOMER ARE SPENDING MORE?
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,SUM(oi.quantity*oi.selling_price) AS revenue
FROM customers c JOIN orders o
ON c.customer_id = o.customer_id
JOIN Order_Items oi
ON o.order_id = oi.order_id GROUP BY c.customer_id,c.first_name,c.last_name ORDER BY revenue DESC LIMIT 1;
-- RAHUL HAS HIGHEST SPENDING AMOUNT

-- TOP 5 CATEGORY HAVE HIGHEST REVENUE
SELECT c.category_name ,SUM(o.quantity*o.selling_price)AS revenue FROM order_items o JOIN
products p ON o.product_id=p.product_id JOIN categories c ON p.category_id=c.category_id GROUP BY c.category_name
ORDER BY revenue DESC LIMIT 5;
-- ELECTRONICS HAVE HIGHEST REVENUE 483490.0

-- WHAT IS LOWEST ORDER VALUE?
SELECT order_id,SUM(quantity*selling_price) AS rev FROM order_items GROUP BY order_id ORDER BY rev ASC LIMIT 1;

```

BUSINESS INSIGHTS

BUSINESS INSIGHTS

1. Customer Insights

Query: Total registered customers.

Insight:

The platform has 50 registered customers, showing a healthy initial customer base.

2. Customer Registration Trend

Query: Customers registered by month.

Insight:

Most customers registered in July, indicating the highest customer acquisition during that month.

3. Product Insights

Query: Highest-priced product.

Insight:

iPhone 15 is the most expensive product priced at ₹79,999, contributing significantly to premium product offerings.

4. Discount Insights

Query: Highest discount.

Insight:

boAt Airdopes offers the highest discount (25%), making it an attractive product for customers.

5. Inventory Insights

Query: Lowest stock products.

Insight:

A few products have lower stock levels and should be monitored for timely replenishment to avoid stock-outs.

6. Shipment Insights

Query: Delivered orders.

Insight:

21 shipments have been successfully delivered, indicating effective order fulfillment.

7. Monthly Shipment Trend

Query: Shipment count by month.

Insight:

July recorded the highest number of shipments, showing peak operational activity.

8. Courier Performance

Query: Courier usage.

Insight:

Courier usage is fairly balanced among delivery partners, reducing dependency on a single logistics provider.

9. Seller Insights

Query: Highest-rated seller.

Insight:

Camera Store has the highest seller rating, reflecting strong customer satisfaction.

BUSINESS INSIGHTS

10. Product Pricing

Query: Average product price by seller.

Insight:

Priya Patel has the highest average product price, indicating a focus on premium products.

11. Wishlist Insights

Query: Most wishlisted products.

Insight:

Eight products appear frequently in customer wishlists, showing strong customer interest and potential future demand.

12. Order Insights

Query: Total orders.

Insight:

The system contains 50 orders, demonstrating active customer purchasing activity.

13. Review Insights

Query: Product ratings.

Insight:

Several products have a 5-star rating, indicating positive customer satisfaction and product quality.

14. Category Insights

Query: Total categories.

Insight:

The platform manages 20 product categories, offering a diverse product catalog.

Business Recommendation

1. Based on all analyses:
2. Increase inventory for frequently purchased products.
3. Promote highly rated products through marketing campaigns.
4. Continue offering competitive discounts on popular products.
5. Strengthen partnerships with the best-performing courier services.
6. Highlight top-rated sellers to build customer trust.
7. Monitor low-stock products to prevent lost sales.

CONCLUSION

CONCLUSION

This project successfully designed and implemented an Amazon E-Commerce Database Management System using PostgreSQL. The database was normalized to efficiently manage customers, products, sellers, orders, payments, shipments, reviews, wishlists, inventory, and other business operations.

Using SQL, various business analyses were performed through joins, aggregate functions, subqueries, CTEs, views, and window functions. These analyses provided insights into customer behavior, product performance, seller performance, shipment trends, inventory status, and overall business operations.

This project demonstrates practical database design skills, SQL query writing, data analysis, and business reporting. It reflects real-world e-commerce scenarios and provides a strong foundation for data-driven decision-making. Future enhancements can include integrating Power BI dashboards, automating reports, and adding advanced analytics for sales forecasting and customer recommendations.

Tab 18

Future Enhancements

Power BI Dashboard
Sales Dashboard
Customer Dashboard
Recommendation System
Fraud Detection

GitHub Repository

Include:

Ecommerce_Database:

<https://github.com/kareenagola97-ui/Amazon-ecommerce-database-management-system>

Business Report :

https://github.com/kareenagola97-ui/Amazon-ecommerce-database-management-system/blob/main/BUSINESS_REPORT.sql